

HR & Employee Info Portal — Project Plan

Web Portal + Flutter Mobile App | Attendance, Leave, HR Ops, Task Management

1. Project Overview

Goal: A unified HR platform (web admin/employee portal + Flutter mobile app) covering the full employee lifecycle — recruitment → onboarding → attendance/time tracking → task management (client/project/task) → leave → HR documentation — with biometric/face/geolocation-based attendance capture.

Primary users:

- **Employees** — mark attendance, view hours, apply for leave, see tasks, events, holidays
 - **Managers** — approve leave, view team attendance/hours, reporting hierarchy view
 - **HR/Admin** — manage CVs, onboarding, documents, org structure, policies, payroll data export
 - **Super Admin** — system config, integrations, roles
-

2. Core Modules

A. Recruitment & CV Management

- Job requisition creation, CV upload/parsing (resume parser), candidate pipeline (Applied → Screened → Interview → Offer → Hired)
- Interview scheduling, scorecards, offer letter generation
- Candidate portal (optional) or shared link for status updates

B. Onboarding

- Digital onboarding checklist (IT setup, documents, policy acknowledgment)
- Document collection (ID proof, education certs, bank details, PAN/Aadhaar for India)
- E-signature for offer letter, NDA, employment contract
- Auto-provisioning trigger (create employee record, assign manager, department, email templates)

C. Employee Master / Profile

- Personal info, employment info, department/designation, reporting manager, documents vault
- Org chart / reporting hierarchy view
- Role-based access control (RBAC)

D. Attendance System (core differentiator)

- **Check-in/out via mobile app** with:

- **Face detection/recognition** (liveness check to prevent photo spoofing)
- **Geolocation capture** + geofencing (mark attendance only within office radius, or flag remote check-ins)
- **Biometric** — this needs a decision (see note below)
- Regularization requests (forgot to check in/out), manual HR override with audit trail
- Shift management, late-mark/early-leave rules, overtime calculation

Important clarification needed on "biometric": Mobile phones don't have a way to capture fingerprint/biometric of *someone else* — phone fingerprint sensors only authenticate the phone owner against their own enrolled fingerprint (device-level, via `local_auth` in Flutter). If you want **fingerprint attendance**, that typically means physical biometric devices (ZKTeco, eSSL, Realtime) installed at office premises, with attendance logs pulled via API/DB sync into your system. Face recognition, on the other hand, works well purely through the mobile camera. I'd recommend: **Face + Geolocation via mobile app** as primary, and **physical biometric device integration** (if you have office premises) as a secondary/parallel data source, unified in the same attendance table. Happy to adjust based on your actual setup — do you have physical office locations with turnstile/biometric hardware, or is this a fully remote/field workforce?

E. Leave Management

- Leave types (casual, sick, earned, comp-off), balance tracking, accrual rules
- Apply/approve workflow with manager notification
- Leave calendar (team view), carry-forward/encashment rules

F. Time & Hours Tracking

- Daily/weekly/monthly hours logged (auto-computed from attendance check-in/out)
- Required hours per month vs. actual (variance/shortfall view)
- Overtime, undertime alerts
- Timesheet view, exportable reports (for payroll)

G. Task Management System (in-house, no Jira dependency)

Hierarchy: **Client → Project → Task**

Client module

- Add/edit client (name, contact person, email, phone, company details, notes)
- One client can have multiple projects
- Client status (active/inactive), client-wise project list view
- **Client is mandatory for every project/task** — no orphan tasks. Internal/admin work (e.g., "Website Revamp - Internal," "Bench/Training," "Sales & BD") is tracked by

creating your own company (Northern Planet / NSSPL) as a client, with its own projects underneath. This keeps effort-tracking, reporting, and utilization numbers consistent whether work is billable or internal — you'll be able to see exactly how many hours go to client-billable work vs. internal overhead, per employee/per month.

- Recommended: flag client records as "**Internal**" vs "**External**" (a simple boolean/type field) so reports can filter/exclude internal client(s) when calculating billable utilization %, without changing the underlying schema.

Project module

- Create project under a client (project name, description, start date, deadline, status, budget/billing type — optional)
- Assign project team members (who can see/work on this project's tasks)
- One project can have multiple tasks
- Project-level progress rollup (% tasks completed, hours logged vs. estimated)

Task module — each task has:

- Task name, detailed description
- Attachments (multiple files — specs, screenshots, references)
- Start time, completed time (auto-stamped or manually set)
- Status: **Pending, In Progress, Hold, Reply Awaited from Client, Completed**
- Priority: **1-9**
- Estimated hours, worked hours (logged)
- Assigned by (who created/assigned the task)
- Assigned to (who's responsible — supports single or multiple assignees)
- Additional recommended fields:
 - Due date / deadline (separate from estimated hours, for scheduling)
 - Task type/tag (bug, feature, support, internal, etc. — configurable)
 - Comments/activity thread (internal discussion on the task, with @mentions)
 - Sub-tasks / checklist items
 - Watchers/followers (get notified without being assigned)
 - Status change log (audit trail — who changed status, when)
 - Time log entries (start/stop timer or manual entry, multiple entries per task, sums to worked hours)
 - Task dependency (optional — "blocked by" another task)
 - Client-visible flag (optional, in case you later want a client-facing view of task status)

Views & features

- Kanban board (by status), list view, calendar view (by due date)
- Filter/sort by client, project, assignee, priority, status, date range
- My Tasks view (personal dashboard — everything assigned to me across projects)
- Manager/Admin view — all tasks across team, workload view (who's overloaded/underloaded based on estimated vs. available hours)
- Overdue task alerts, SLA/reply-awaited aging (e.g., flag if "Reply Awaited from Client" for >X days)
- Reports: time spent per client/project/employee, estimated vs. actual hours variance, task completion rate, **billable (external client) vs. internal utilization** % per employee/team/month (using the Internal/External client flag)

H. Employee Dashboard (Home)

- Today's attendance status, hours this week/month vs. required
- My Tasks summary (pending, in progress, overdue, due today) pulled from the Task Management module
- Upcoming events, public holidays, birthdays/work anniversaries
- Pending leave approvals (for managers), announcements/notices
- Quick links: apply leave, regularize attendance, view payslip, log task time

I. HR Admin Console

- Employee lifecycle management (transfers, promotions, exit/offboarding)
- Document repository (policies, letters, templates)
- Holiday calendar management, event/announcement posting
- Reports: attendance summary, leave summary, headcount, attrition

J. Notifications

- Push (mobile), email, in-app — leave approvals, task deadlines, holiday reminders, missed check-in alerts

K. Reporting & Analytics

- Attendance trends, department-wise hours, leave utilization
- Exportable reports (Excel/PDF) for management/payroll processing

L. (Optional, Phase 2) Payroll-adjacent

- Export hours/leave data to payroll system (not full payroll processing unless scoped separately)
 - Exit management, F&F settlement tracking
-

3. Third-Party Integrations Needed

Category	Options	Notes
Face recognition/liveness	AWS Rekognition, Azure Face API, Google ML Kit (on-device, free), FaceIO, Kairos	ML Kit is on-device/free but basic; AWS/Azure give liveness + higher accuracy, cost per API call
Geolocation/geofencing	Google Maps Platform (Geocoding + Geofencing), native <code>geolocator</code> (Flutter)	Google Maps API has usage-based pricing
Physical biometric devices (if needed)	ZKTeco, eSSL, Realtime Biometrics — via SDK/API or DB polling	Only if office-based fingerprint devices are in scope
Push notifications	Firebase Cloud Messaging (FCM) — free, standard for Flutter	
E-signature	DocuSign, Zoho Sign, SignEasy	For offer letters, NDAs, contracts
Resume parsing	Affinda, RChilli, Sovren, or open-source (spaCy-based)	Speeds up CV screening
Calendar sync	Google Calendar API, Outlook/Microsoft Graph API	For events/holidays sync to personal calendars
Email/SMS	SendGrid/Mailgun (email), Twilio/MSG91 (SMS/WhatsApp — MSG91 good for India)	
Document storage	AWS S3 / Google Cloud Storage / Azure Blob	For CVs, certificates, signed documents
Payroll (India)	Zoho Payroll, RazorpayX Payroll, GreytHR (export/API)	If not building payroll in-house
HRIS/Background verification (optional)	SpringVerify, AuthBridge	Pre-employment checks
Analytics	Firebase Analytics / Mixpanel (app usage), custom dashboards	

Category	Options	Notes
	for HR data	
SSO (Phase 2)	Google Workspace SSO, Microsoft Entra ID	If employees use company Google/O365 accounts

4. Suggested Tech Stack

- **Mobile app:** Flutter (iOS + Android from single codebase) — `camera`, `google_mlkit_face_detection` or vendor SDK, `geolocator`, `local_auth` (for device-level biometric auth to the app itself, not attendance biometric), `firebase_messaging`
- **Web portal (Employee + HR Admin):** React or Next.js (given your team's existing stack), Tailwind
- **Backend:** Node.js (NestJS/Express) or Laravel (if your team is stronger there) — REST/GraphQL API
- **Database:** PostgreSQL (relational, good for reporting/joins across attendance/leave/employee data)
- **File storage:** AWS S3 or GCS
- **Auth:** JWT-based, refresh tokens, RBAC middleware
- **Hosting/Infra:** AWS/GCP, containerized (Docker), CI/CD via GitHub Actions

5. Data Privacy & Compliance (Important)

Since this captures **face/biometric + location data** of employees:

- Under India's **DPDP Act 2023**, biometric data is treated with heightened care — requires explicit consent, purpose limitation, and secure storage (encryption at rest).
- Store raw face images minimally — prefer storing **face embeddings/vectors** (not raw photos) where possible, with photos auto-purged after verification.
- Maintain a clear **consent capture flow** during onboarding (employee explicitly consents to biometric/location tracking for attendance purposes).
- Location tracking should be **limited to check-in/check-out moments**, not continuous background tracking — this avoids both privacy issues and battery drain complaints.
- Have a documented data retention & deletion policy.

This is worth 30 minutes with a lawyer or compliance consultant before launch — biometric data mishandling carries real regulatory risk in India now.

6. Phased Development Plan

Phase 0 — Discovery & Design (2-3 weeks)

- Finalize exact attendance mechanism (face-only vs. face+physical biometric device)
- Wireframes/UI design (web + Flutter)
- Data model design, API contract design
- Third-party vendor selection (face SDK, SMS/email, storage)
- Task management data model design (Client → Project → Task, status/priority schema, time-log structure)

Phase 1 — Foundation (3-4 weeks)

- Auth, RBAC, employee master module
- Org structure, reporting hierarchy
- Admin console shell, web portal shell, Flutter app shell

Phase 2 — Attendance Core (4-5 weeks)

- Flutter: camera + face capture + liveness, geolocation capture, check-in/out flow
- Backend: attendance engine, shift rules, regularization workflow
- Hours calculation (daily/weekly/monthly vs. required)

Phase 3 — Leave Management (2-3 weeks)

- Leave types, balances, accrual, apply/approve workflow, leave calendar

Phase 4 — Recruitment & Onboarding (4 weeks)

- CV upload/parsing, candidate pipeline, interview scheduling
- Onboarding checklist, document collection, e-signature integration

Phase 5 — Task Management System + Dashboard (5-6 weeks)

- Client module, Project module, Task module (full CRUD + all fields listed above)
- Kanban/list/calendar views, time logging against tasks, workload view
- Dashboard widgets (my tasks, hours, events, holidays)
- Notifications (push/email — task assigned, status changed, overdue, reply-awaited aging)

Phase 6 — Reporting, Polish, QA (3 weeks)

- Reports/exports, analytics dashboards
- Load testing (especially face API calls at check-in peak times, e.g., 9-9:30 AM)
- Security audit (esp. biometric/location data handling)
- UAT with a pilot group of employees

Phase 7 — Launch & Stabilization (2 weeks)

- Phased rollout (pilot team → org-wide)
- Bug fixes, feedback loop, performance tuning

Estimated total: ~25-29 weeks (6-7 months) for a full-featured v1, assuming a small dedicated team (2 backend, 2 frontend/Flutter, 1 QA, 1 PM/designer). Building task management in-house (vs. a Jira pull) adds real scope — it's effectively its own product — but gives you full control over workflow, no per-seat licensing cost, and no external dependency. Timeline compresses if you trim scope (e.g., defer recruitment module) or extends if biometric hardware integration is involved.

7. Suggested MVP Scope (if you want to launch faster)

If you want a faster first release, I'd suggest cutting to:

1. Employee master + RBAC
2. Attendance (face + geolocation only, no physical biometric device)
3. Hours tracking + dashboard (hours/day/week/month + required hours)
4. Leave management
5. Holidays/events display
6. Task Management core (Client → Project → Task, statuses, priority, estimated/worked hours, assignment — skip Kanban/calendar views, dependencies, and workload analytics for v1)

Defer to v2: Recruitment/CV management, e-signature onboarding, physical biometric device integration, payroll export, analytics dashboards, task dependencies, workload view, SLA aging alerts.

This gets a usable internal tool live in **~10-12 weeks**, and you validate the attendance mechanism (the riskiest/most novel part) before building the heavier HR-ops modules on top.

Open Questions Before Finalizing Scope

1. Do you have physical office premises (for geofencing radius + potential biometric hardware), or is this for a distributed/field team?
2. Is "biometric" strictly fingerprint, or is face recognition sufficient to satisfy that requirement?
3. Do you want payroll processing built in, or just data export to an existing payroll tool?
4. Roughly how many employees will use this (affects DB/infra sizing and face-API cost estimates)?

5. Do you want this built by your own Northern Planet team, or are you scoping this to quote out/pitch as a product?